Green DevOps – Energy Efficient CI/CD Pipeline

R26-SE-027

Project Proposal Report

Rathnayake D.M.B.H – IT22149930

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Rathnayake. D.M.B.H.	**IT22149930**	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mr.Jagath Wickramarathna	2026.03.11	

ABSTRACT

The ICT sector currently contributes approximately 2-4% of global CO2 emissions, a footprint comparable to the aviation industry [1]. The rapid expansion of cloud-native architectures and high-frequency DevOps pipelines has led to a significant "green debt," where software efficiency is frequently sacrificed for delivery speed and developer productivity [2] [1] [3].

Current cloud provider tools have a measurement gap, meaning they cannot accurately track important environmental factors such as embodied carbon and network transfer impacts [1]. Because software development has traditionally prioritized developer speed over resource efficiency, many organizations have accumulated “green debt,” which refers to environmental inefficiencies built into existing systems that are now difficult and expensive to fix [1].

Popular CI/CD tools rarely treat environmental performance as a key metric and generally lack built-in carbon tracking or sustainability KPIs [3]. Additionally, many existing AI solutions operate as isolated tools rather than integrated systems, which limits their ability to provide real-time feedback or effectively connect DevOps practices with environmental science [2].

Researchers describe an optimization trilemma where delivery speed, system reliability, and environmental sustainability often conflict [1] [4]. Although studies show that nearly 90% of CPU time on CI workers is wasted, there is a lack of multi-criteria decision systems to balance these factors [5]. The problem is further compounded by the absence of standardized green benchmarks, making it difficult for teams to evaluate and compare the sustainability of different software architectures [6] [1] [3].

This research proposes an AI-driven SusDevOps framework specifically for the deployment stage [6]. The solution features a granular energy profiler to calculate SCI and an intelligent deployment strategy and time decision engine for carbon-aware scheduling, which aligns deployment tasks with renewable energy windows

The framework uses machine learning models to analyze historical system logs and Python-based scripts to monitor real-time resource usage [2] [7]. It leverages Kubernetes orchestration to schedule workloads on clusters that produce lower carbon emissions [8].

The expected results include a 30-45% reduction in energy consumption and a 25-40% decrease in carbon emissions while maintaining system performance and service-level objectives [2]. This approach helps integrate sustainability into DevOps practices, promoting Green DevOps [1].

In addition, the framework can reduce cloud operational costs by minimizing over-provisioned resources [2]. It also contributes to global climate action and SDGs by encouraging environmentally responsible digital infrastructure management [6].

Keywords: SusDevOps, Green DevOps, Carbon-Aware Scheduling, Optimized Pipeline, Resource Optimization.

ACKNOWLEDGEMENT

I want to express my sincere gratitude to my supervisor, **Mr. Jagath Wickramarathna**, for his valuable guidance, constructive feedback, and continuous support throughout the preparation of this research proposal. His expertise and encouragement significantly shaped the direction and quality of this work.

I would also like to extend my appreciation to my co-supervisor, **Ms. Madusha Weerasooriya**, for her insightful suggestions and support during the development of this research proposal.

My sincere thanks also go to our external supervisor, **Mr. Dilanka Nayanajith**, for his industry insights, professional advice, and valuable guidance, which helped us better understand the practical aspects of the research.

Furthermore, I would like to thank the academic staff of the **Sri Lanka Institute of Information Technology (SLIIT)** for providing the necessary knowledge and resources that supported the preparation of this proposal.

Finally, I would like to express my appreciation to my fellow team members **Mendis J.D.L.**, **Thassara M.P.M.**, and **Kodithuwakku I.P.** for their collaboration, discussions, and teamwork throughout the early stages of this research.

I am also grateful to everyone who contributed directly or indirectly to the successful preparation of this research proposal.

Table of Contents

DECLARATION 2

ABSTRACT 3

ACKNOWLEDGEMENT 4

LIST OF TABLES 7

LIST OF FIGURES 8

LIST OF ABBREVIATIONS 9

LIST OF APPENDICES 10

1. INTRODUCTION 11

1.1 Background and Context 11

1.2 Literature Review 11

1.3 Research Gap 12

2. OBJECTIVE 13

2.1 Main Objective 13

2.2 Specific Objectives 13

3. METHODOLOGY 14

3.2 Technical Methods and Algorithms 14

3.3 Tools and Platforms 16

3.4 Data Collection and Ethical Considerations 16

3.5 Validation Strategy and Performance Metrics 17

4. HIGH-LEVEL SYSTEM ARCHITECTURE 19

4.1 Overview of the Proposed Solution and Its Components 19

4.2 Highlighting the Individual Component – Deploy Stage 20

5. USER REQUIREMENTS 23

5.1 Stakeholder Identification 23

5.2 Functional Requirements 23

5.3 Non-Functional Requirements 24

6. COMMERCIALIZATION PLAN 25

6.1 Target Market and Customer Persona 25

6.2 Value Proposition 25

6.3 Revenue Model 25
6.4 Cost Estimation 26
6.5 Pricing Strategy 26
6.6 Competitive Advantage 27
6.7 Intellectual Property Considerations 27
7. BUDGET AND JUSTIFICATION 28
8. WORK BREAKDOWN STRUCTURE (WBS) 30
9. GANTT CHART 32
10. REFERENCES LIST 33
11. APPENDICES 34

LIST OF TABLES

Table 1: List Of Abbreviations 9
Table 2: Commercialization Cost Estimation Table 26
Table 3: Budget Table 28
Table 4: Work Breakdown Structure Table 31

LIST OF FIGURES

Figure 1: XGBoost Algorithm Simple Diagram 15
Figure 2: High-level System Diagram..... 20
Figure 3: Deploy Stage Component Architecture 22
Figure 4: Gantt Chart 32

LIST OF ABBREVIATIONS

Abbreviation	Full Form
SusDevOps	Sustainable Development and Operations
DevOps	Development and Operations
CI/CD	Continuous Integration / Continuous Deployment
GCP	Google Cloud Platform
AWS	Amazon Web Services
TOSCA	Topology and Orchestration Specification for Cloud Applications
EDMM	Essential Deployment Meta Model
DORA	DevOps Research and Assessment
ICT	Information and Communication Technology
IT	Information Technology
ID	Identifier
OT	Operational Technology
OCI	Oracle Cloud Infrastructure
KPI	Key Performance Indicator.
UN	United Nations
SGD	Sustainable Development Goals
SCI	Software Carbon Intensity
AI	Artificial Intelligence
ML	Machine Learning
FREEDA	Failure-resilient, energy-aware, and explainable deployment of microservice-based applications over Cloud-IoT infrastructures
TOSCA	Topology and Orchestration Specification for Cloud Applications
EDMM	Essential Deployment Meta Model
kWh	A kilowatt-hour
CPU	Central Processing Unit

Table 1: List Of Abbreviations

LIST OF APPENDICES

Appendices 1: Deployment Energy Profiler Architecture..... 32
Appendices 2: Deployment Decision Engine Architecture 32
Appendices 3: Green Deployment Component Architecture 35

1. INTRODUCTION

1.1 Background and Context

The environmental footprint of information technology is rapidly expanding, especially within energy-intensive industries like utility operations [1]. Modern DevOps relies heavily on automated delivery pipelines that consume significant computational resources, contributing to a growing "green debt" where software efficiency is neglected in favor of delivery speed [1]. This problem affects organizations through rising operational costs and society at large due to increased greenhouse gas emissions [1]. Addressing this is critical as sustainability becomes a strategic imperative for financial viability and regulatory compliance [1].

Globally, the ICT sector contributes approximately 2.4% of all greenhouse gas emissions, while data center energy consumption is reaching unprecedented levels due to AI expansion [1] [2]. Locally, organizations have reported a 35% annual increase in cloud spending often linked to inefficient build processes and over-provisioned resources [3]. Current high-performance DevOps standards, like DORA metrics, frequently ignore environmental impact [1]. A primary limitation is the "measurement gap," where common tools lack the granularity to track energy or carbon metrics, leaving developers unaware of their environmental footprint [1] [4].

The proposed solution focuses on a "SusDevOps" or Green DevOps framework that treats sustainability as a first-class concern [5] [3]. Key components include carbon-aware scheduling to time deployments when grid intensity is low and the use of AI/ML models to predict and optimize energy usage across the pipeline. This research directly aligns with the United Nations' Sustainable Development Goals by aiming to protect the planet while meeting current needs [5] [6]. The work is situated within the research clusters of Sustainable Software Engineering (SSE) and Green IT [5] [2].

1.2 Literature Review

Existing Research Approaches

Carbon-Aware Scheduling: This approach delays non-urgent deployment jobs until the electrical grid's carbon intensity drops, reducing emissions without impacting delivery performance [4] [2].
Trade-off Modeling: Research like the FREEDA toolchain uses a multi-criteria solver to plan deployments by finding a balance between conflicting requirements, such as failure resilience and energy consumption [7].
Metric Adaptation: Existing performance standards, like DORA metrics, are being adapted to incorporate environmental considerations, such as measuring kg CO2 per deployment [1].
Continuous Reasoning: New techniques are being developed to timely adapt the deployment of applications when changes occur in the infrastructure or sustainability requirements [7].

Existing Products and Services

Energy Monitoring Tools: Practical tools such as the Green Metrics Tool, Eco CI, Scaphandre, and Energy ID are used to measure energy consumption within CI/CD pipelines [1].
Cloud Provider Calculators: Major platforms offer services like the Azure Sustainability Calculator and GCP Carbon Footprint Tool to help organizations track and report environmental performance [2].
Optimization Frameworks: Cloud Carbon Footprint is an open-source tool that measures compute usage in terms of kilograms of CO2 emitted [2].

Declarative Modeling: Tools like TOSCA and EDMM enable DevOps engineers to enact energy-optimized service placements across Cloud-Edge nodes [7].

Limitations of Existing Systems

The "Measurement Gap": Popular CI/CD platforms provide runtime data but frequently omit energy or carbon metrics, leaving developers unaware of their environmental footprint [4] [1].
Incomplete Data: Existing environmental data often lacks granular detail regarding embodied carbon and network transfer impacts [1].
Conflict with Resilience: Strategies to increase failure resilience, such as service replication, often result in higher energy consumption, creating a conflict that many current systems cannot resolve [7].
Single-Aspect Focus: Most existing deployment solutions focus on single metrics like cost or hardware requirements rather than a holistic, explainable approach to sustainability [7].
Legacy Integration Issues: There is significant difficulty connecting new Green DevOps tools to old, often proprietary, IT and OT systems used in critical infrastructure [1].
Organizational Bias: Teams often prioritize delivery speed and pressure over sustainability, leading to a "green debt" where software efficiency is neglected [4] [1].

1.3 Research Gap

There is a critical absence of a holistic framework that integrates environmental sustainability as a first-class concern within the automated DevOps lifecycle [1] [2]. Specifically, current research lacks standardized green benchmarks and metrics tailored for different software domains, leaving teams without a clear way to evaluate their environmental footprint [5] [2]. Furthermore, there is a shortage of practical mitigation strategies that teams can implement within full real-world pipelines rather than just small-scale prototypes [4].

Existing high-performance standards, such as DORA metrics, focus exclusively on speed and stability, which can lead to environmentally detrimental "high-performing" pipelines that consume excessive energy through unoptimized builds [1]. Current monitoring tools suffer from a "measurement gap," frequently providing runtime data but omitting essential factors like embodied carbon and network transfer impacts [1]. Additionally, existing deployment techniques are often single-aspect, focusing only on cost, hardware, or service dependencies rather than a multi-criteria approach that includes sustainability [7].

This research moves beyond traditional direct hardware profiling by proposing the use of Machine Learning prediction models to estimate the energy footprint of automated workflows [4] [2]. It introduces a three-dimensional view of performance that forces a balance between speed, stability, and ecological stewardship [1]. Innovation is further driven by the application of "continuous reasoning" techniques, which allow for the timely adaptation of deployments when infrastructure conditions or sustainability requirements change [7].

The heavy reliance on automation in modern DevOps carries significant environmental costs, as every process from code integration to release consumes computational energy [4]. Organizations have reported 35% annual increases in cloud spending due to inefficient build processes and over-provisioned resources [3]. This accumulation of "green debt" occurs because software efficiency has historically taken a backseat to developer productivity and delivery velocity [1]. Establishing a sustainable DevOps (SusDevOps) framework is therefore essential to prevent the growing negative impact of the IT industry on the planet [5].

2. OBJECTIVE

2.1 Main Objective

The main objective is to develop and evaluate an AI-driven SusDevOps framework for the deployment stage that reduces the environmental footprint of software releases by integrating real-time energy profiling and carbon-aware decision-making

2.2 Specific Objectives

Establish a granular monitoring system to capture resource utilization and estimate SCI. The framework utilizes monitoring scripts to collect real-time data on CPU usage, memory consumption, and deployment duration. This data is essential because software design choices directly influence hardware utilization and subsequent energy consumption. These metrics are integrated into the SCI formula, which calculates "Carbon per Unit of work" by combining energy consumption (E), grid carbon intensity (I), and hardware embodied carbon (M). This shifts accountability to developers by allowing them to quantify the environmental impact of their specific code and algorithms.

Develop a predictive Machine Learning model to estimate the energy footprint of automated workflows. Research indicates a clear shift from direct hardware-based profiling to the use of ML prediction models for measuring energy consumption in CI/CD pipelines. The proposed framework uses AI to analyze historical deployment logs and utilization patterns to provide actionable recommendations for energy reduction. These ML models can predict the environmental cost of different architectural decisions and coding practices before they are executed. This predictive approach helps identify energy inefficiencies that are often "baked in" during the design phase.

Design a carbon-aware scheduling algorithm that dynamically times deployments based on grid carbon intensity. Carbon-aware scheduling involves adapting deployment timing based on real-time feeds of the electrical grid's carbon intensity. Computationally intensive tasks, such as complex simulations or large-scale deployments, can be strategically delayed to run when renewable energy generation is high. By integrating carbon intensity APIs, the pipeline can automatically route workloads to cleaner compute regions or off-peak hours. This allows organizations to reduce their operational carbon footprint without compromising delivery speed for urgent changes.

Implement a multi-criteria decision engine to select energy-optimized deployment strategies. Achieving sustainability is a complex multi-objective optimization problem that requires balancing speed, cost, and environmental impact. The framework uses a decision engine to find a trade-off between conflicting requirements, such as maintaining failure resilience while minimizing brown energy consumption. This engine evaluates different strategies such as canary or blue-green deployments to identify "win-win" scenarios where environmental benefits align with operational reliability. It also accounts for the energy cost of "re-work" caused by high change failure rates.

3. METHODOLOGY

Research Approach The research follows a Design Science and Experimental approach focused on creating two primary artifacts: a Deployment Energy Profiler for measurement and an AI Decision Engine for optimization. This approach addresses the technical and environmental impact of unmonitored deployment pipelines by establishing a feedback loop for sustainability. The system perspective is built on an Input-Process-Output model, where real-time resource data is processed to generate green deployment plans.

3.2 Technical Methods and Algorithms

Energy Profiling

A Python-based monitoring script leverages the psutil library to capture system-level metrics during deployments. Key metrics include average CPU usage, memory consumption, and deployment duration. These metrics form the basis for estimating energy consumption in cloud virtual machines, where direct measurement is not feasible.

Estimation Formulas

Energy consumption is calculated using the standard formula:

$$\text{Energy (kWh)} = \text{Power (W)} \times \text{Time (hours)}$$

Power is estimated as:

$$\text{Power} = \text{Idle Power} + (\text{CPU Utilization} \times \text{Dynamic Power})$$

This approach allows the system to approximate the energy used for each deployment based on real-time CPU and memory usage.

Machine Learning Algorithm

The AI Decision Engine employs XGBoost, a supervised learning algorithm, to predict deployment energy consumption and associated carbon emissions. Supervised learning works by training the model on historical deployment data, where the input features are system metrics (CPU, memory, deployment time) and the target variables are energy and carbon output. XGBoost is chosen for its high accuracy, robustness to overfitting, and ability to handle complex, non-linear relationships in the data. [12]

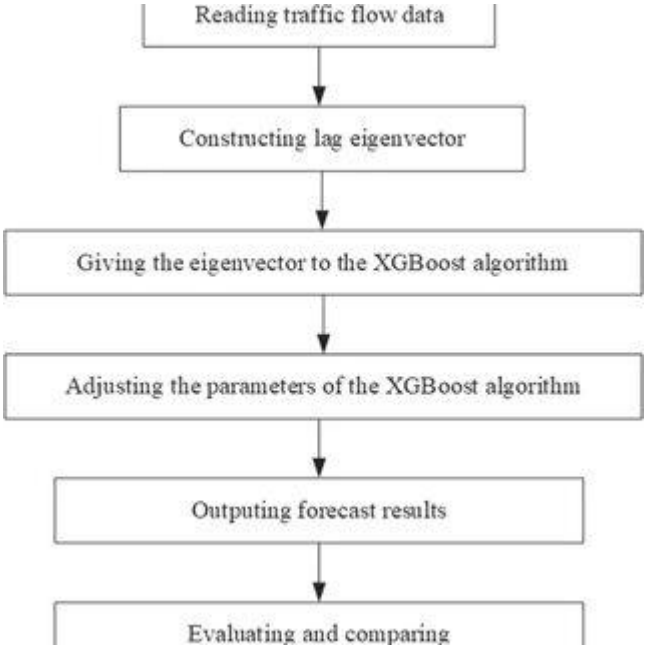


Figure 1: XGBoost Algorithm Simple Diagram

Decision Logic and Deployment Strategies

The system evaluates multiple deployment strategies to determine the most environmentally efficient approach

Rolling Deployment: Gradually updates a subset of servers at a time, reducing peak resource usage and minimizing downtime.

Canary Deployment: Releases changes to a small group of users or servers first, monitoring resource impact before full rollout.

Blue-Green Deployment: Maintains two identical environments (blue and green), switching traffic to the new environment after deployment to minimize disruption.

3.3 Tools and Platforms

Category	Tools / Platforms	Purpose in the Framework
Cloud Infrastructure	Oracle Cloud Infrastructure (OCI)	Provides compute instances to run monitoring scripts and deployment analysis.
Data Management	Oracle Autonomous Database, Oracle Object Storage	Stores historical deployment logs, energy metrics, and model training datasets.
CI/CD Platform	Jenkins, GitHub Actions	Integrates the framework into existing CI/CD pipelines to monitor deployment processes.
Pipeline Language	Groovy	Used to define and manage Jenkins pipeline scripts for automated deployments.
Development Language	Python	Used to develop monitoring scripts, energy estimation models, REST APIs, and ML workflows.
Containerization	Docker	Packages deployment environments and services to ensure consistent execution across infrastructure.
Machine Learning Model	XGBoost	Used to predict deployment energy consumption and recommend optimized deployment timing.
Learning Approach	Supervised Learning	Trains the model using historical deployment data to learn patterns in energy usage and carbon emissions.

3.4 Data Collection and Ethical Considerations

The proposed framework relies on both real-time system data and publicly available datasets to analyze deployment energy consumption and estimate carbon emissions.

Primary Data

Primary data is collected directly from the deployment environment during the execution of the CI/CD pipeline. The monitoring scripts capture real-time system metrics such as CPU utilization percentage, virtual memory usage percentage, and deployment timestamps. These metrics help measure how much computing power is used during each deployment. By combining resource

usage with the duration of the deployment process, the system can estimate the energy consumed by the infrastructure during the deployment stage.

External Data
In addition to the real-time metrics, the framework also uses external datasets to support model training and carbon emission estimation. The machine learning model is trained using public Azure-related energy datasets available through GitHub repositories. These datasets provide historical information about cloud infrastructure energy usage patterns. Furthermore, regional carbon intensity data is used to convert estimated energy consumption into carbon emission values. This allows the system to estimate the environmental impact of each deployment in terms of kilograms of CO_2 produced.

Ethical Usage
The framework is designed with responsible data usage in mind. The monitoring system only collects infrastructure-level metrics related to the deployment process and does not access any personal or sensitive user data. All logs used for analysis are anonymized system logs, ensuring that no identifiable information is stored or processed. Additionally, the system focuses strictly on the deployment stage of the CI/CD pipeline, meaning application-level data, developer data, or end-user information is not monitored. By relying on anonymized logs and publicly available datasets, the framework maintains ethical standards and protects user privacy.

3.5 Validation Strategy and Performance Metrics

To evaluate the effectiveness of the proposed Green DevOps deployment framework, specific performance indicators and validation methods are used to measure environmental improvements during the deployment stage.

Performance Metrics

The main performance indicators used to measure the success of the framework are energy consumption per deployment and carbon emissions per deployment.

- **Energy used per deployment (kWh):**
  This metric measures the total amount of electrical energy consumed during a single deployment process. It is calculated based on monitored infrastructure resource usage such as CPU utilization, memory consumption, and deployment duration. Lower energy usage indicates a more efficient deployment process.

- **Carbon emission per deployment (kg CO_2):**
  This metric estimates the environmental impact of each deployment by converting the energy consumption into carbon emissions using regional carbon intensity data. It represents the amount of carbon dioxide produced as a result of the electricity used during deployment.

Together, these metrics help determine whether the deployment process is environmentally efficient and sustainable.

Validation Method

The framework is validated by comparing the environmental impact of deployments performed under different strategies. Specifically, the system predicts carbon emissions for multiple possible deployment windows using historical deployment data and real-time infrastructure metrics.
The machine learning model analyzes these factors and recommends the most environmentally efficient deployment time, referred to as the "green deployment plan."
To verify the effectiveness of the framework, two scenarios are compared.

- **Traditional Deployment Approach**
  Deployments are executed immediately without considering energy usage or carbon intensity.

- **Green Deployment Approach**
  Deployments are scheduled based on the framework's recommendation, selecting time windows with lower predicted energy consumption and carbon emissions.

By comparing the energy consumption and carbon emissions of these two approaches, it becomes possible to determine whether the proposed framework successfully reduces environmental impact.

Using **real deployment metrics and historical CI/CD logs** ensures that the validation process reflects realistic deployment conditions, improving the reliability and credibility of the results.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed Solution and Its Components

The proposed system introduces sustainability awareness into the CI/CD pipeline, specifically focusing on the deployment stage, where infrastructure resources are heavily utilized. Traditional DevOps pipelines prioritize speed and reliability [1], but they do not consider energy consumption or carbon emissions during deployment operations [1] [7]. The proposed solution integrates a monitoring and decision-making layer into the CI/CD lifecycle to measure deployment energy usage and make environmentally optimized deployment decisions. The architecture consists of four major components:

Release Component
Processes sustainability insights to optimize release preparation by avoiding unnecessary work and enabling carbon-aware scheduling.

Deploy Component
Executes deployment operations while measuring deployment energy usage and selecting energy-efficient deployment strategies using AI.

Operate Component
Monitors deployed applications during runtime and optimized infrastructure operations through carbon-aware scaling and resource management.

Monitoring Component
Collects sustainability-related metrics such as energy consumption, carbon intensity, and CI/CD execution data.

The system creates a continuous sustainability feedback loop where operational data influences future deployment decisions.

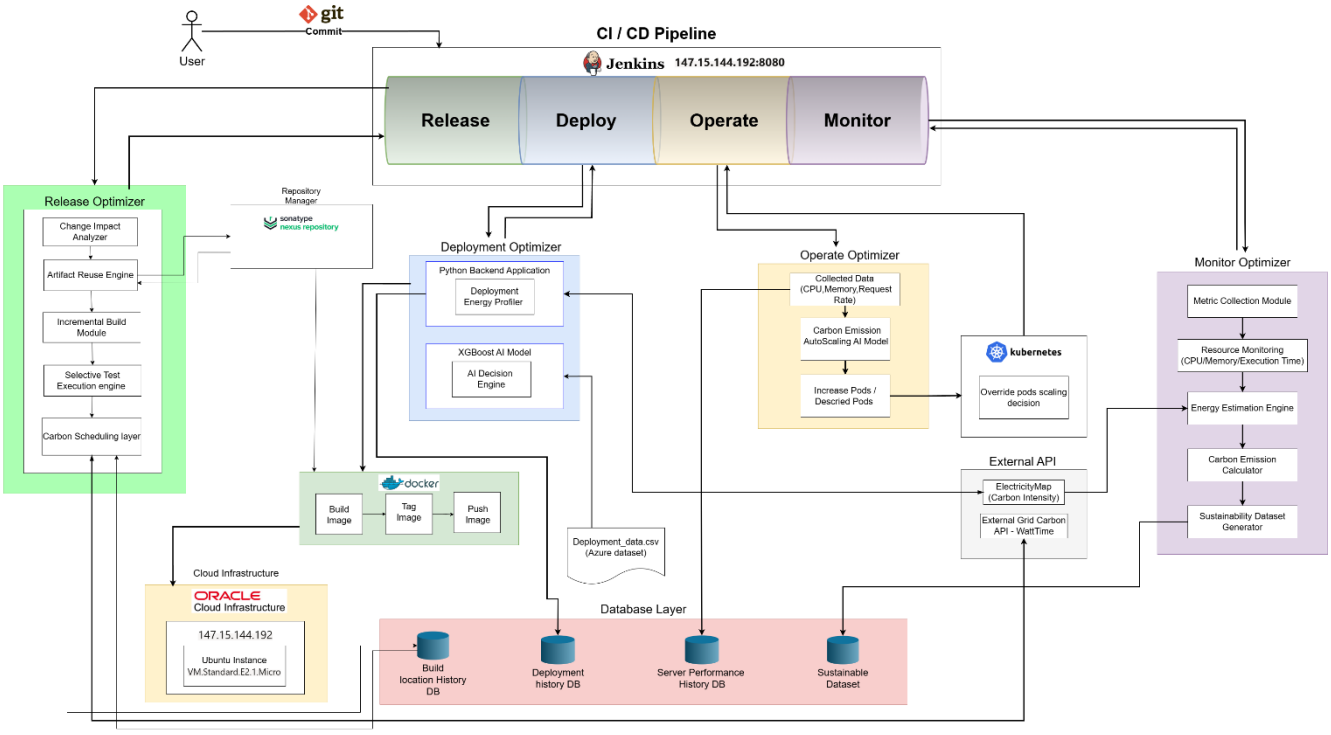


Figure 2: High-level System Diagram

4.2 Highlighting the Individual Component – Deploy Stage

This research primarily focuses on the Deploy Component of the proposed Green DevOps architecture. The deployment stage is selected because software deployments involve intensive infrastructure usage, including compute resources, networking operations, and container orchestration activities. As a result, deployments contribute to measurable energy consumption and associated carbon emissions within cloud-based environments.

The Deploy Component introduces sustainability-aware intelligence into the deployment pipeline by incorporating two key subcomponents: the Deployment Energy Profiler and the AI-Based Green Deployment Decision Engine. Together, these components enable both measurement and optimization of deployment-related environmental impact.

Deployment Energy Profiler

The Deployment Energy Profiler is responsible for monitoring and estimating the energy consumption and carbon emissions generated during the deployment process. Accurate measurement of resource utilization during deployment is essential because it establishes the baseline data required for sustainable optimization.

During each deployment execution, the profiler collects several infrastructure-level metrics related to resource usage. These metrics include:

- CPU utilization during the deployment process
- Memory utilization of the deployment environment

- Deployment duration, representing the total time required to complete the deployment
- CI/CD execution metrics, such as pipeline stage duration and system workload

These collected metrics provide an approximation of the computational effort required for each deployment.

Using the monitored resource usage data, the profiler estimates energy consumption per deployment using widely accepted data center power estimation models. In such models, system power consumption is approximated as a function of CPU utilization and execution time, since CPU usage is strongly correlated with server power consumption in typical cloud infrastructure environments.

Following the energy estimation step, the system calculates the carbon emissions associated with the deployment. Carbon emissions are estimated by multiplying the calculated energy consumption with the regional carbon intensity factor, which represents the amount of carbon dioxide (CO_2) emitted per unit of electricity consumed within a given energy grid.

The results generated by the profiler are stored as deployment energy records, forming a historical dataset that includes information about resource usage, estimated energy consumption, and corresponding carbon emissions for each deployment event.

This dataset serves as the primary input for the AI-based decision component, enabling data-driven optimization of future deployments.

AI-Based Green Deployment Decision Engine

The AI-Based Green Deployment Decision Engine introduces intelligent decision-making into the deployment process by analyzing historical deployment energy data and predicting environmentally efficient deployment configurations.

This component utilizes supervised machine learning, specifically the XGBoost algorithm, to identify patterns between deployment characteristics and their associated energy consumption and carbon emissions. XGBoost is selected due to its high predictive performance, efficiency in handling structured datasets, and widespread use in performance-sensitive predictive modeling tasks.

The machine learning model evaluates multiple deployment-related features derived from historical data, including:

- CPU utilization patterns observed during previous deployments
- Deployment duration
- Deployment size, representing the scale or complexity of the release
- Time of day, which may influence infrastructure load and energy conditions
- Deployment urgency level, indicating whether the deployment can be delayed
- Deployment strategy type, such as rolling, canary, or blue-green deployment

Using these features, the trained model predicts the expected energy consumption and carbon emissions associated with different deployment scenarios.

Before executing a deployment, the decision engine evaluates possible deployment configurations and compares their predicted environmental impact. Based on this evaluation, the system selects the deployment configuration that minimizes energy consumption and carbon emissions while maintaining operational reliability and service availability.

The decision engine generates two key outputs that guide the deployment process:

- Recommended deployment timing, identifying an optimal time window for execution
- Recommended deployment strategy, such as rolling deployment, canary deployment, or blue-green deployment

These outputs enable the deployment system to perform environmentally optimized deployments, reducing the carbon footprint of the software delivery process without negatively affecting system stability or release performance.

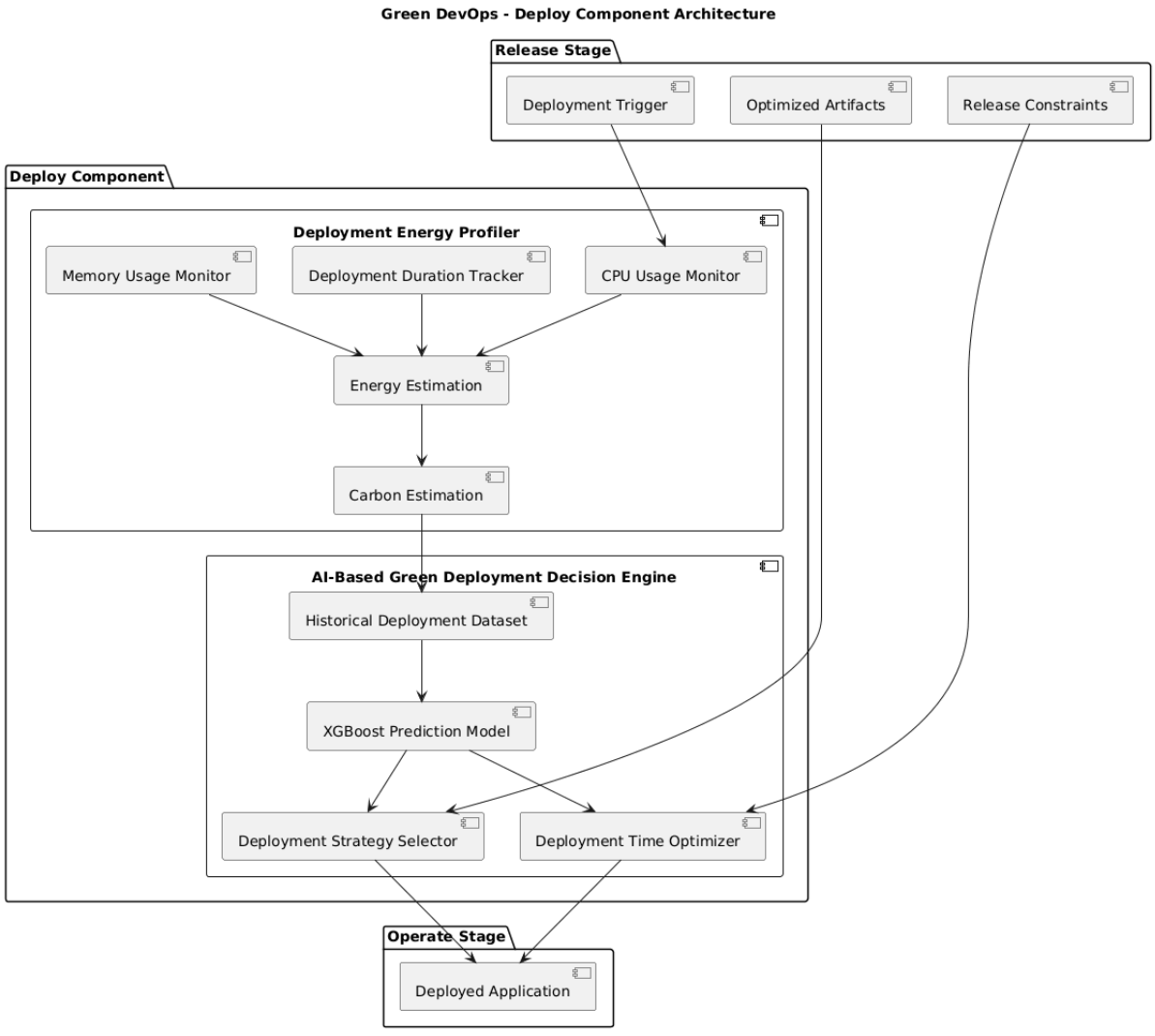


Figure 3: Deploy Stage Component Architecture

5. USER REQUIREMENTS

5.1 Stakeholder Identification

- Organizations and Infrastructure Owners: Stakeholders include organizations operating large-scale cloud or data-center infrastructures that face rising operational costs and the need for regulatory compliance [1].
- DevOps and Site Reliability Engineers: These professionals are responsible for release decisions, application installation, and monitoring, directly influencing how software utilizes computational resources [3].
- Cloud Service Providers: Providers are affected by escalating energy demands and have a vested interest in sustainability commitments, such as carbon neutrality [3].
- Society and Regulatory Bodies: Society at large is affected by increased greenhouse gas emissions from ICT, while regulators drive the adoption of green practices through legal mandates.

5.2 Functional Requirements

- Granular Resource Profiling: The system must established monitoring scripts to capture average CPU usage, memory usage, and deployment duration in real-time [1].
- Environmental Impact Estimation: The framework must estimate the Software Carbon Intensity (SCI) by combining estimated energy consumption, grid carbon intensity, and hardware embodied carbon [1].
- Predictive Modeling: The system must utilize Machine Learning prediction models to analyze historical logs and estimate the energy footprint of automated workflows before execution [4].
- Carbon-Aware Scheduling: The component must be able to dynamically delay or trigger deployments based on real-time feeds of electrical grid carbon intensity to prioritize renewable energy windows [2].
- Multi-Criteria Strategy Selection: The engine must evaluate and select the most energy-efficient strategy (e.g., Rolling, Canary, or Blue-Green) while maintaining operational reliability [3].
- CI/CD Pipeline Integration: The solution must integrate with standard automated pipelines via REST APIs to allow for continuous compilation and deployment optimization [6] [1].

5.3 Non-Functional Requirements

- Performance and Latency: Any "green" optimization implemented cannot introduce significant performance overhead, as grid stability and service continuity are non-negotiable priorities [2] [1].
- Security and Integrity: Environmental efficiency measures must not introduce new vulnerabilities or compromise the resilience of critical national infrastructure [1].
- Observability and Usability: The system must provide visual sustainability indicators (e.g., kg CO2 per build) within existing DevOps dashboards to make environmental impact tangible and actionable for teams [4] [1] [2].
- Reliability and Fault Tolerance: The framework must support failure-resilient deployments, ensuring that reducing brown energy consumption does not negatively impact the quality of service [7].
- Scalability: The framework must be capable of handling microservice-based applications across large-scale, heterogeneous Cloud-Edge computing continuums [7] [2].
- Cultural Alignment: The solution should support a shift toward "SusDevOps", where environmental responsibility is treated as a core quality attribute of software engineering alongside speed and stability [5] [2].

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

Organizations operating large-scale cloud or data-center infrastructures and cloud service providers facing rising energy demands.

Customer Persona:

DevOps Engineers: Responsible for release decisions and looking for ways to optimize deployment pipelines.

Sustainability Officers: Seeking to reduce the organizational environmental footprint and meet carbon neutrality goals.

Infrastructure Managers: Focused on reducing operational costs in "pay-as-you-use" cloud environments.

6.2 Value Proposition

The proposed solution provides visibility and accountability by addressing the measurement gap in DevOps processes, offering real-time insights into energy consumption and carbon emissions specifically during the deployment stage.

By analyzing this data, the system helps reduce environmental impact by intelligently selecting environmentally friendly deployment windows and applying energy-efficient deployment strategies such as rolling, canary, and blue-green deployments to minimize greenhouse gas emissions.

In addition to environmental benefits, the approach can generate economic savings by reducing unnecessary resource usage and avoiding deployments during high-energy-intensity periods, which lowers operational costs.

Furthermore, the system introduces intelligent optimization by moving beyond manual scheduling and leveraging AI to automatically determine the most suitable time and strategy for deployments based on real-time carbon intensity and infrastructure conditions

6.3 Revenue Model

The revenue model is based on multiple streams. First, a SaaS subscription model will be offered with tiered pricing depending on the number of monitored deployment pipelines or the frequency of deployments. In addition, enterprise licensing will be provided for organizations that require custom integration with private data centers or have specific security requirements.

Furthermore, consultancy and integration services will be offered to assist organizations in integrating the Deployment Energy Profiler and AI Decision Engine into their existing Jenkins or GitHub Actions workflows.

6.4 Cost Estimation

The cost structure includes several key components. Infrastructure costs involve the usage of Oracle Cloud (OCI) resources, including compute instances for running monitoring scripts and Oracle Autonomous Database or Object Storage for storing historical logs.

Development and maintenance costs cover the ongoing work required to refine the XGBoost supervised learning model and update carbon intensity data feeds. Additionally, data acquisition costs may arise from accessing high-granularity regional carbon intensity APIs that provide more detailed information than standard global averages.

Category	**Description**	**Estimated Cost (LKR)**
Development Cost	AI model development, system architecture, software implementation	1,200,000
Infrastructure & Hardware	Cloud clusters, prototype hardware (Raspberry Pi, sensors)	500,000
Maintenance & Marketing	System maintenance, promotion, deployment support	750,000
Total Launch Cost		**2,450,000 LKR**

Table 2: Commercialization Cost Estimation Table

6.5 Pricing Strategy

The pricing strategy includes several approaches to accommodate different types of users. Pay-per-deployment allows small-scale pricing for teams with a low deployment frequency.

Usage-based tiering provides scaled pricing for high-performance teams that require continuous profiling across multiple microservices. In addition, value-based pricing is applied based on the documented reduction in carbon emissions or energy costs achieved through the use of the system.

6.6 Competitive Advantage

The proposed system offers several unique advantages. It enables holistic decision making by not only measuring energy consumption but also determining both the optimal time to deploy and the most energy-efficient deployment strategy.

It provides real-time technical depth through live Python-based monitoring of CPU usage, memory consumption, and deployment duration instead of relying on simple static estimates.

The system also ensures seamless CI/CD integration by operating as a microservice with a REST API that can easily integrate into standard pipelines such as Jenkins or GitHub Actions.

Additionally, it incorporates predictive intelligence by using the XGBoost model to simulate deployment scenarios and predict energy consumption before execution, enabling proactive and environmentally conscious deployment planning.

6.7 Intellectual Property Considerations

The intellectual property considerations include several key aspects. The proprietary AI logic refers to the unique decision-making mechanism that balances deployment urgency with carbon intensity trends and deployment strategy selection.

The energy estimation algorithms involve specialized formulas designed to estimate cloud virtual machine power usage in situations where direct hardware energy measurement is not possible.

In addition, software copyright applies to the unique Python implementation of the Deployment Energy Profiler and the integration framework developed for Oracle Cloud.

7. BUDGET AND JUSTIFICATION

The following budget outlines the estimated costs required to develop, test, and demonstrate the **Deployment Energy Profiler** and the **AI-Based Green Deployment Decision Engine**.

Budget Table (Approximate Prices in LKR)

Category	**Item**	**Quantity**	**Estimated Price (LKR)**
Hardware	**Raspberry Pi 3** (Demo Hardware)	1	10,000.00
	DHT22 (Temp) Sensor	1 Set	1,500.00
	SD Card & Power Supply cable		1,000.00
Cloud Services	**Oracle Cloud Infrastructure (OCI)** (Compute & DB)	1 Year	Using free tier
Software	Open Source (Python, XGBoost, psutil, pandas)	N/A	0.00
	GitHub Actions / Jenkins		Using free ones
Human Effort	Student	1	200,000.00
Total Estimated Budget			**212,500.00**

Table 3: Budget Table

Justification for Each Expense

Hardware Costs (Raspberry Pi & Sensors):

As noted in the sources, direct energy measurement is impossible in cloud VMs. The Raspberry Pi will be used for a physical demo to capture real-time temperature and power consumption during deployment. This serves as a tangible proof-of-concept for how software deployment directly impacts hardware heat and energy usage.

Software Licenses:

The core technical implementation relies on open-source tools like **Python, psutil** for monitoring, and **XGBoost** for the predictive decision engine. Willing to use free sources for **CI/CD pipeline** (GitHub/Jenkins) to support the frequent automated triggers needed to build the historical energy dataset.

Cloud Services (OCI):

The framework is designed to run on **OCI Compute Instances**. This infrastructure is essential for the AI engine to analyze historical patterns and time-based trends. Choose Oracle Because it is free tier.

Human Effort Cost:

Significant human effort is required for Step 1 (Monitoring Scripts) to collect CPU and memory data, Step 2 (Energy Estimation) using standard formulas, and Step 3 (ML Training) to develop the XGBoost model that predicts the carbon output of different strategies.

Demonstration (Temp & Carbon):

The demo requires a physical setup to bridge the "measurement gap" identified in the sources. By using a Raspberry Pi, the system can provide a live display of carbon emissions per deployment alongside physical temperature changes, validating the "green deployment plan".

8. WORK BREAKDOWN STRUCTURE (WBS)

WBS ID	Task	Description	Timeline
D2.0	Project Planning	Initial planning and understanding of the Green DevOps deployment optimization concept	Dec 2025
D2.1	Literature Review	Study research papers on Green DevOps, carbon-aware computing, and energy-efficient deployments	Feb 2026
D2.2	Problem Definition	Identify the deployment energy measurement gap and define project objectives	Feb 2026
D2.3	Proposal Preparation	Prepare research proposal including methodology and system architecture	Early Mar 2026
D2.4	Proposal Presentation	Present the research proposal to the evaluation panel	16th – 18th Mar 2026
D2.5	System Design	Design the architecture of the Deployment Energy Profiler and AI Decision Engine	Mar 2026
D2.6	Deployment Energy Profiler Development	Implement Python scripts to monitor CPU usage, memory usage, and deployment duration	Mar – Apr 2026
D2.7	Data Collection	Gather deployment logs and carbon intensity data required for analysis	Mar – Apr 2026
D2.8	Data Preprocessing	Clean and structure the collected dataset for machine learning training	Apr 2026
D2.9	AI Model Development	Build the predictive model for deployment energy optimization	Apr – May 2026
D3.0	Model Training	Train the XGBoost supervised learning model using the processed dataset	Apr – May 2026
D3.1	Prediction Logic Implementation	Implement the decision logic for selecting optimal deployment time and strategy	May 2026
D3.2	System Integration	Integrate monitoring, prediction, and CI/CD pipeline components	May – Jul 2026
D3.3	API Development	Develop REST API to allow CI/CD tools to communicate with the system	May – Jun 2026
D3.4	CI/CD Integration	Integrate the system with Jenkins or GitHub Actions pipelines	Jun – Jul 2026
D3.5	Evaluation & Optimization	Test and improve system performance and prediction accuracy	Aug – Sep 2026
D3.6	Performance Testing	Evaluate prediction accuracy and energy estimation reliability	Aug 2026
D3.7	System Optimization	Improve model performance and energy estimation algorithms	Sep 2026

WBS ID	Task	Description	Timeline
D3.8	Documentation & Final Deliverables	Prepare final outputs for evaluation	Sep – Oct 2026
D3.9	Final Report Writing	Complete detailed project report documentation	Sep – Oct 2026
D4.0	Website Development	Develop project website and documentation portal	Sep – Oct 2026
D4.1	Final Presentation & Viva	Demonstrate the system and present results	12th – 14th Oct 2026
D4.2	Research Paper Preparation	Write and submit research paper based on findings	Oct 2026

Table 4: Work Breakdown Structure Table

9. GANTT CHART

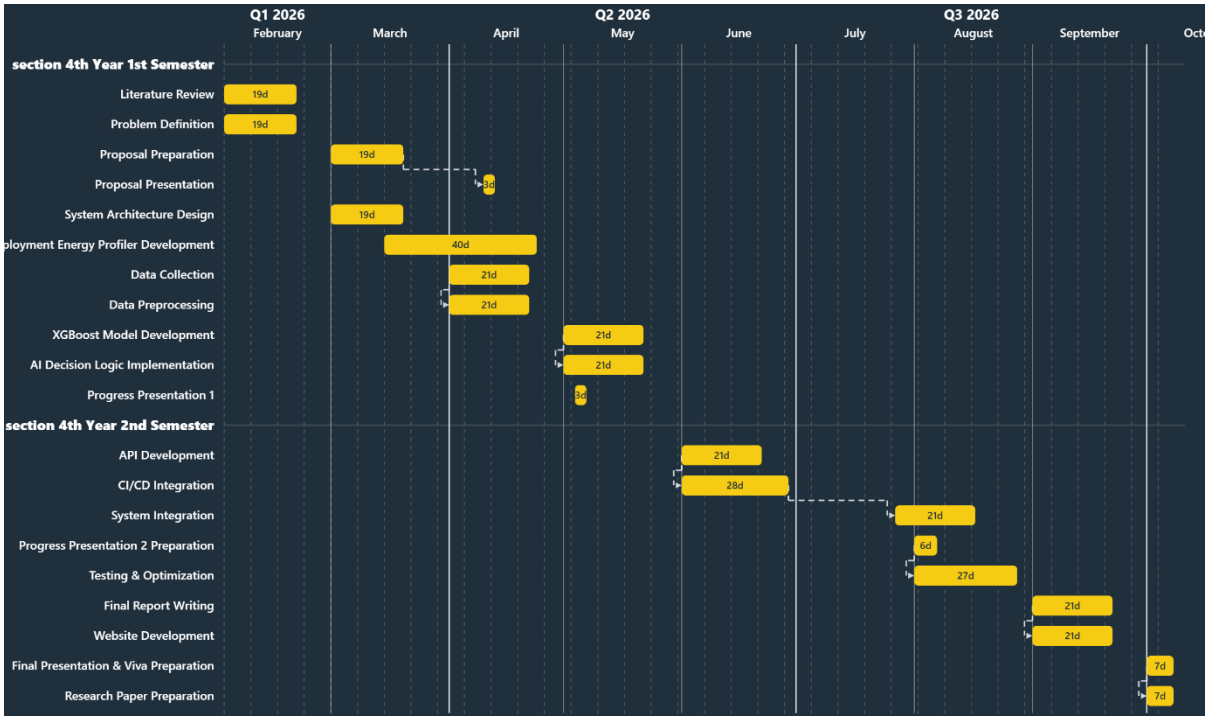


Figure 4: Gantt Chart

10.REFERENCES LIST

[1] L. P. Rongali, "Green DevOps Metrics for Utility Operations," *Green DevOps Metrics for Utility Operations,* 2025.

[2] Michael Okpotu Onoja, Cynthia Chidinma Onyenze and Akeem Amusa Akintoye, "DevOps and Sustainable Software Engineering: Bridging Speed,," *International Journal of Technology Management & Humanities (IJTMH),* vol. 10, no. 4, pp. 60-80, 2024.

[3] H. Allam, "Sustainable Cloud Engineering: Optimizing Resources," *International Journal of Artificial Intelligence, Data Science, and Machine Learning ,* vol. 4, no. 4, pp. 36-45, 2023.

[4] "Sustainable Devops: A Systematic Literature Review on Reducing Energy Footprint in Continuous Integration and Deployment (Ci/Cd) Pipelines," *International Journal of Computations, Information and Manufacturing (IJCIM),* pp. 24-32, 2025.

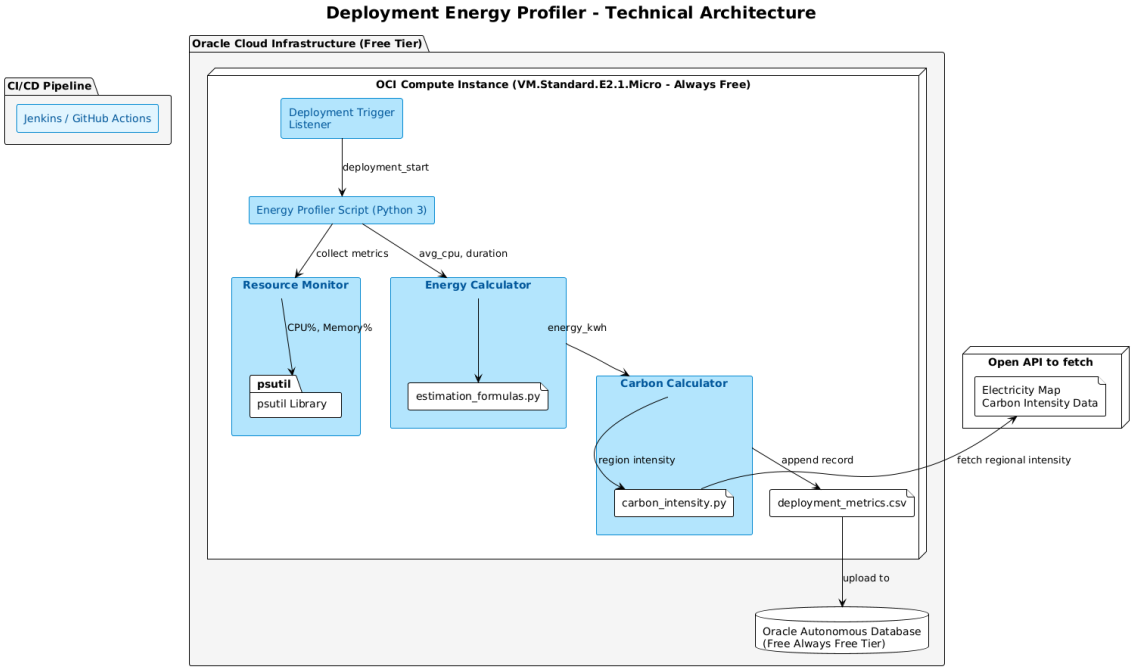
[5] Darwish Ahmad Herati, Maria Clara Aderne and Fabio Kon, "The road to Sustainable DevOps," *The road to Sustainable DevOps,* pp. 1-4, 2025.

[6] Nitin Kumar and Dr Neeraj Sharma, "A Comprehensive Study forGreen Computing-Based DevOps Process," *UGC CARE Listed (Group -I) Journal,* vol. 11, no. 11, pp. 2820-2828, 2022.

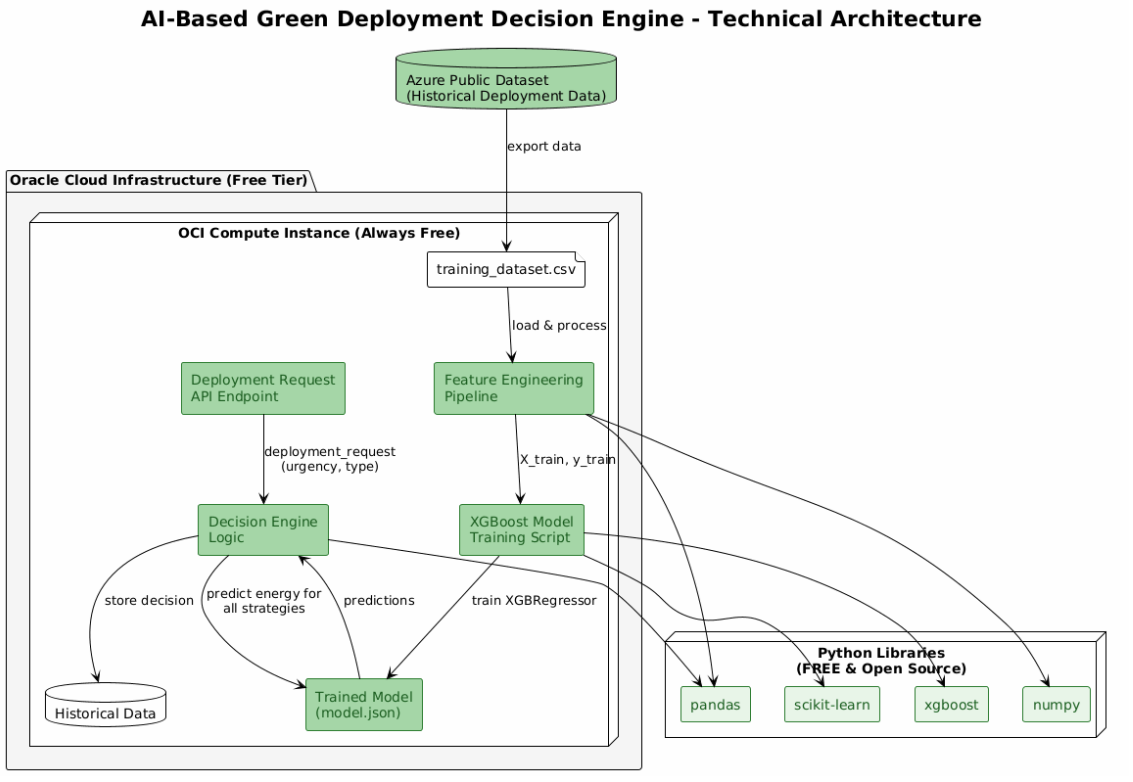
[7] Jacopo Soldani, Roberto Amadini, Antonio Brogi, Stefano Forti, Saverio Giallorenzo, Pierluigi Plebani, Monica Vitali and Gianluigi Zavattaro, "Towards Sustainable Deployment of Microservices over the Cloud-IoT," *Towards Sustainable Deployment of Microservices over the Cloud-IoT,* pp. 1-5, 2024.

[8] A.Retalis, "Geographic information systems and science," *The Photogrammetric Record,* p. 396, 2005.

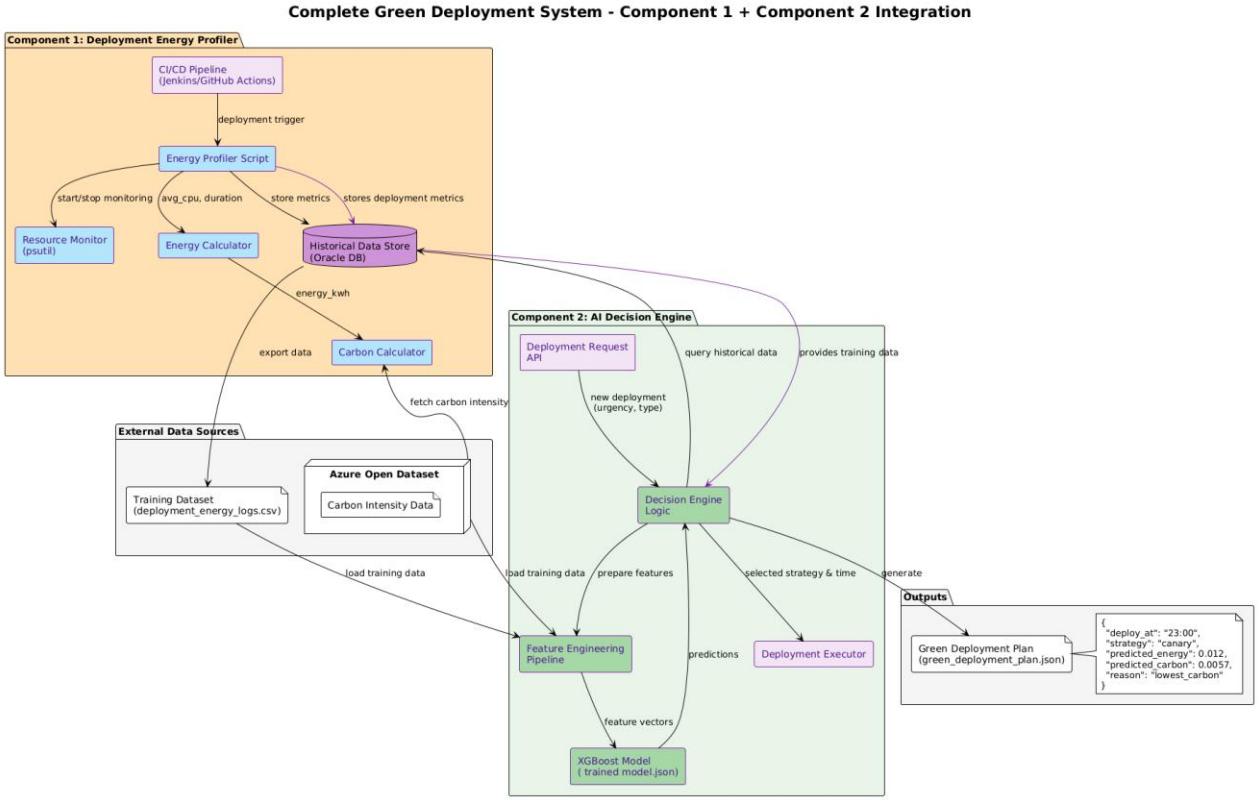
11.APPENDICES



Appendices 1: Deployment Energy Profiler Architecture



Appendices 2: Deployment Decision Engine Architecture



Appendices 3: Green Deployment Component Architecture